Ders 8 Genişletilmiş Notu: Refactoring'e Giriş ve JUnit ile Birim Sınaması

Ders 8 Genişletilmiş Notu: Refactoring’e Giriş ve JUnit ile Birim Sınaması

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Refactoring Nedir, Ne Amaçlar?
- 2. Refactoring Nasıl ve Ne Zaman Yapılır?
- 3. Code Smell: Kötü Kokan ya da Kusurlu Kod
- 4. Birim Sınaması
- 5. JUnit ve JUnit 3.x ile Test Case Hazırlamak
- 6. JUnit 4.x Gelişmeleri
- 7. Video Kiralama Örneği: Tasarım
- 8. Video Kiralama Örneği: Kodun İlk Hali
- 9. Video Kiralama Örneğinin JUnit Testleri
- 10. Başarısız Bir Testin İncelenmesi ve Test Suite
- Örnek Sorular
- Dersin Toparlaması

1. Refactoring Nedir, Ne Amaçlar?

Tasarım kalıplarının ardından ders, “code smell ve refactoring” konusuna geçer. Hoca bu derste konunun temelini ve girişini yapacağını, ayrıntılı kod kusurlarına sonraki derslerde geçileceğini söyleyerek başladı.

1.1 Tanım

Slayttaki ilk tanım şudur: bir kod parçasının **ne yaptığını değiştirmeden nasıl yaptığını değiştirmeye** refactoring denir (ekrandan, 00:06:00). Daha dar ikinci tanım: yazılımın **dışarıdan görünen davranışı değiştirilmeden iç yapısının değiştirilmesi**.

Hoca “dışarıdan görünen” ifadesine dikkat çekti. Dışarıdan bakan yalnız bir insan kullanıcı değildir; sistemle tümleşik çalışan başka yazılımlar da olabilir. Bu durumda sistemin dışarıya açılan erişim arayüzü, yani sınıflar, sınıf adları, sınıf imzaları ve metot adları da korunması gereken davranışın parçası sayılabilir. Dar anlamda refactoring bu arayüze de dokunmaz.

1.2 Amaç: iç kalite

Refactoring ile amaçlanan, mevcut kod üzerinde kodun **iç kalite özelliklerini** daha iyiye götürecek değişiklikler yapmaktır. Hoca dersin ilk haftasındaki kalite konusunu hatırlattı: iç kalite özellikleri programcıyı ve yazılım ekibini, dış kalite özellikleri ise son kullanıcıyı ilgilendirir. Ne yapıldığını değiştirmeden nasıl yapıldığını değiştirmek, doğası gereği iç kaliteye odaklanmaktır.

Slayttaki amaç listesi şöyledir (ekrandan, 00:06:00):

- Kodun anlaşılabilirliğinin artırılması
- Koda esneklik kazandırılması
- Yeniden kullanılabilirliğin artırılması
- Yüksek uyum ve düşük bağımlılığın sağlanması
- vb.

Hoca bunlardan ilk ikisini en önemlileri olarak gördü. **Anlaşılabilirlik** bakım masrafını düşürür. Bakım, yazılım sahaya çıktığı andan itibaren üzerinde yapılan her türlü değişikliktir: hata düzeltme, gereksinim değişikliklerini aktarma ya da yeni gereksinim ekleme. Kod ne kadar kolay anlaşılırsa bakım da o kadar kolay yürür. Hocaya göre bir yazılım projesinde harcanan emeğin yaklaşık %50'si bakım aşamasında harcanır; yani yazılım sahaya çıkınca iş bitmez, ancak yarısına gelinmiş olur. **Esneklik** de yine bakım kolaylığı, yani değişikliklerin daha kolay yapılması anlamına gelir.

Yeniden kullanılabilirlik için yapılan refactoring sahada çalışan kodun üzerinde yapılmayabilir; ekibin kendi kod kitaplığının bir sürümü üzerinde yapılır. Bu sürüm sonra sahadaki diğer yazılımlara aktarılabilir ya da aktarılmayabilir. Amaç, belli kod kitaplıklarını başka projelerde de kullanarak daha az çabayla daha çok iş yapmaktır. Son madde ise en temel iki kalite göstergesi olan bağlaşım ve uyumla ilgilidir.

2. Refactoring Nasıl ve Ne Zaman Yapılır?

2.1 Nasıl yapılır?

Slayta göre refactoring iki ilkeyle yapılır (ekrandan, 00:10:00):

- **Çevik yaklaşımla yapılır:**
  - Mevcut gereksinimleri karşılayacak ilk tasarım yapılır.
  - İlk tasarımın mükemmel olması ve her bilinmezi öngörmeye çalışması gerekmez.
  - Yeterince iyi olan tasarım gerçeklenmeye başlanır.
  - Önceki tasarım adımlarından birinde yanlış karar verildiği anlaşılınca refactoring yapılır.
- **Sürekli sınama ile yapılır:**
  - Yapılan değişikliklerin yeni hatalar oluşturmadığından emin olmak için her değişikliğin ardından kodun yeni hali sınanmalıdır.

Hoca ikinci ilkeyi daha önemli buldu: refactoring eyleminin ardından sisteme istemeden çeşitli hatalar sokulmuş olabilir. "Her değişiklik" derken her satırdan sonra sınama kastedilmiyor; bir miktar değişiklikten sonra sınama yapılmalıdır. Kaç küçük adımdan sonra kapsamlı bir sınama yapılacağına (buna regresyon sınaması denir) geliştirici karar verir. Hocanın önerisi şudur: genelde bir sınıf üzerinde çalışılır ve bir sorumluluk birkaç metodun birlikte çalışmasıyla yerine getirilir; bir sorumluluk tamamlandığında o sorumluluk sınanabilir. Sınama da bir yazılım projesinde zamanı ve çabası planlanan bir etkinlik olduğu için ayrılan zamana sığacak şekilde mümkün olduğunca sık tekrarlanmalıdır.

2.2 Ne zaman yapılır?

Slayta göre kod derlenip toparlanmaya gerek duyduğu zaman refactoring yapılır (ekrandan, 00:10:00):

- Koda yeni özellikler eklemekte zorlanmaya başlandığında,
- Bir hatayı bulmak için zorlanmaya başlandığında, bulunduğunda da düzeltmek için birçok ayrı yeri kurcalamak zorunda kalındığında,
- vb.

Hoca bunun ölçülebilir bir işaretini de verdi: bir proje yönetim yazılımı kullanılıyorsa ve hatalar küçük, orta, büyük diye sınıflanıyorsa, aynı büyüklükteki işlerin tamamlanma süresinin ilk zamanlara göre gitgide uzadığı fark edilebilir. Bir proje yöneticisi ya da kalite birimi bu noktada refactoring yapılmasına karar verebilir.

Refactoring'i yalnız bakım aşamasındaki bir değişiklik olarak düşünmemek gerekir. Yazılım en baştan çevik bir süreçle geliştiriliyorsa, örneğin her Scrum döngüsünün yeni bir özellik eklemesi (bir increment olması) gerekmez; bir döngü kodu toparlamaya ayrılabilir.

Bütün bu işaretler mecazi bir deyişle tek cümleye bağlanır: **kod kötü kokmaya başladığında refactoring yapılır.**

3. Code Smell: Kötü Kokan ya da Kusurlu Kod

3.1 Tanım

Slayttaki tanımlar şöyledir (ekrandan, 00:14:20):

- Yazılan kodu iyileştirme gerektiğinin işaretleridir.
- Kod içerisinde karşılaşılan tersliklerdir.
- Kullanılan programlama yaklaşımının kusurlu kullanım örnekleridir.
- Zamanla bir yazılım yamalı bohçaya dönüşebilir:
  - Değişik zamanlarda farklı kişilerce kodun değişik kısımlarında değişiklikler yapılır.
  - Bu değişiklikler birlikte düşünülerek planlanabilseydi daha iyi bir çözüme ulaşılması mümkün olabilirdi.
- Yazılım elimizden kayıp gitmeden tehlike işaretlerini sezip önlem almak yerinde olacaktır.

İngilizcede bir şeyin "koktuğunu" söylemek genelde kötü koku anlamına gelir. Hoca "code smell" için Türkçede "kusurlu kod" demeyi tercih ettiğini açıkladı. Gerekçesi şudur: code smell'ler birer hata (bug) değildir. Program yanlış çalışmaz; ama verimsiz çalışmaya ya da düzenleme yapmayı zorlaştırmaya yol açarlar. Yani kod işini yapar, ama olabilecek en iyi şekilde yapmaz; "biraz kusurlu yapar". Hoca adlandırma konusunda öğrencilerin önerisi varsa dinlemeye açık olduğunu da ekledi.

"Yamalı bohça" benzetmesi de hata düzeltme kodlarına "yama" (patch) denmesinden gelir: bir yerde hata bulunur, oraya yama atılır; başka yerde bulunur, oraya da atılır. Yapılması gereken bütün değişiklikler önceden öngörülebilseydi en akıllıca biçimde yapılırdı; ama bu mümkün olmaz ve değişiklikler farklı kişilerce farklı zamanlarda yapılır. Böylece tutarsızlıklar ve kusurlar ortaya çıkar. Bunların hepsi bakım masraflarını artıracak risklerdir.

3.2 Code smell ve refactoring ilişkisi

Slayta göre (ekrandan, 00:14:20):

- Kodda karşılaşılan terslikleri düzeltmek üzere yeniden yapılandırma eylemleri yürütülür.
- Bir yapılandırma eylemi yeni kokular çıkartabilir; bu durumda yeni yapılandırma eylemleri yürütülür.

Bu yüzden refactoring adım adım yürütülen bir eylemdir. Bütüncül bir yeniden tasarım değildir; kodda yapılan küçük küçük değişiklik ve düzeltmelerdir. İlerleyen derslerde farklı kod kusurları ve her birini gidermek için gereken refactoring eylemleri görülecektir.

Hoca "code smell" için "kusurlu kod" karşılığını önerse de "refactoring" için iyi bir Türkçe karşılık bulamadığını söyledi. "Yeniden yapılandırma" ifadesi finans alanındaki borç yapılandırmayı da çağrıştırıyor; hoca bu karşılığın kavramı tam karşılamadığını düşünüyor.

4. Birim Sınaması

Refactoring birim sınamaları eşliğinde yapılacağı için hoca konuya birim sınamasıyla devam etti.

4.1 Tanım ve ne zaman tasarlandığı

Slayta göre (ekrandan, 00:20:00):

- Birim sınaması (unit testing) en küçük yazılım bileşeninin sınanmasıdır.
  - NYP'de bireysel sınıfların sınanmasıdır.
- Ne zaman tasarlanır? Kodlamadan önce (TDD), kodlama sırasında veya kodlamanın ardından.

Birinci seçenekte daha tek satır uygulama kodu yazılmadan sınama kodları tasarlanır. Buna Test Driven Design ya da Test Driven Development denir. Bu yaklaşımın savı şudur: bir yazılım

biriminin yalnız ihtiyaç duyulan düzeyde yeterli ve doğru sınamasını hazırlayamıyorsanız, büyük olasılıkla o kısmı yanlış tasarlamışsınızdır. Çevik yaklaşımlarda TDD önerilir: önce birim sınamaları hazırlanır, sonra yalnız bu sınamaları geçmeye yönelik kod yazılır. Böylece kod ile sınamalar birbiriyle uyumlu olur.

Böyle çalışmak şart değildir. Bir kısım kod yazıp bir kısım test vakası hazırlanabilir ya da bir birim tamamen kodlandıktan sonra sınama kodları yazılabilir. Hocaya göre bunların hepsi kabul edilebilir; yeter ki sınama yapılsın ve her şey tamamlandıktan sonraya, şelale modelindeki gibi en sona bırakılmasın.

> **Hocanın vurgusu:** Hataların mümkün olduğunca erken tespiti önemlidir. Yazılım geliştirme sürecinde ilerledikçe değişiklik ve düzeltme yapmanın masrafı üstel olarak, katlanarak artar.

4.2 Vekil kod

Bir sınıfın tek başına yürütemediği, başka sınıfların nesneleriyle birlikte çalışarak yürüttüğü sorumluluklar olabilir. Birlikte çalışılacak sınıflar henüz kodlanmamışsa sınama bekletilmek zorunda değildir. Slayta göre, bir sınıfın tek başına yürütemediği sorumlulukların sınanması için bu sınıfın ihtiyaç duyduğu diğer sınıfların yerine geçecek kod gerekebilir (ekrandan, 00:20:00):

- Vekil, sahte, yalancı kod/sınıf vb. (stub, dummy, surrogate, proxy)
- Vekil, sadece ihtiyaç duyulan sınıflar gerçeklenene dek kullanılır.

Bu kod derleme hatası vermeyecek kadar minimum bir iskelettir. Hocanın örneği: başka bir sınıfta, bir denetim yapıp sorun olup olmadığını boolean olarak döndüren bir metot gerekiyorsa, gövdesine basitçe `return true;` yazılır. Böylece o tarafta hata çıkmadığı varsayılır ve kendi tarafımızdaki kodun birim sınamaları yürütülür.

Hoca bu terimler arasında nüanslar olduğunu, bunun başlı başına bir uzmanlık konusu olduğunu ve bölümde yazılım kalitesi ve test teknikleri üzerine ayrı bir ders bulunduğunu belirterek ayrıntıya girmedi.

5. JUnit ve JUnit 3.x ile Test Case Hazırlamak

5.1 JUnit hakkında

Slayta göre (ekrandan, 00:20:00):

- JUnit, Java ile yazılmış kodun birim sınamaları için bir çerçeve programdır (framework).
- Diğer diller için de sürümleri bulunmaktadır; örneğin C# için csUnit.
- IDE desteği: Eclipse'te IDE ile hazır gelir (build path → add libraries); NetBeans'te de IDE ile hazır gelir.

Bu aile genel olarak xUnit diye anılır; x yerine konan harfe göre dile özgü sürüm adlanır. İstenirse JUnit'in en güncel sürümü ayrıca indirilip kütüphaneleri projeye eklenebilir; hoca IDE ile hazır gelenleri kullandığını söyledi. Ders JUnit'in iki sürümü üzerinde durur: 3 ve 4. Sürüm 4 geriye doğru uyumludur ama bazı ek işlevler getirir.

5.2 JUnit 3.x yapısı

Slayttaki sınıf şemasının metne dökümü şöyledir (ekrandan, 00:30:00):

```
paket junit.framework
  TestCase
    # setup()
  Assert
    + assertEquals(beklenen, sonuc : Object)
    + assertNotEquals(Object, Object)
    + assertTrue(boolean)
    + assertFalse(boolean)
    + assertNull(Object)
    + assertNotNull(Object)
```

```
  TestCase - - -> Assert  (bağımlılık)

paket myPackage
  AppClass1, AppClass2              (sınanacak uygulama sınıfları)
  TestAppClass1 (+ testXXX())  --|> TestCase,  - - -> AppClass1
  TestAppClass2 (+ testYYY())  --|> TestCase,  - - -> AppClass2
  TestAll (+ suite() : Test)    ---> TestAppClass1, TestAppClass2
```

Slayttaki açıklamalar:

- Her sınıfın birim testi için ayrı sınıfta ayrı test case'ler hazırlamak, ardından test case'leri bir test suite altında toplamak tercih edilmiştir.
  - TestAll sınıfını yazmak zorunlu değildir; TestAppClass sınıfları bireysel olarak da çalıştırılabilir.
  - `TestAll.suite()` metodu statiktir.
  - `TestCase.setup` metodu her `testXXX()` için yeniden çalışır.

JUnit 3, kendi test sınıflarımızı `TestCase` sınıfından kalıtımla türetmemizi ister. AppClass1'i sınayan sınıfa TestAppClass1, AppClass2'yi sınayan sınıfa TestAppClass2 gibi adlar verilir. Bu test sınıflarındaki her bireysel birim sınamasının, yani test metodunun adı `test` kelimesiyle başlamalıdır. Hoca `test1`, `test2`, `test3` gibi adlar verilmemesini istedi: sınama zaten zor bir iştir; metodun adından neyin hangi şartlar altında sınandığı anlaşılabilmelidir.

> **Ders sonrası not:** Hoca JUnit 3'te test sınıfı adlarının da `test` ile başlamasının zorunlu olduğunu, yoksa sınıfın test case sayılmadığını söyledi (00:24:02). JUnit 3'ün zorunlu tuttuğu, büyük olasılıkla, yalnız test metodu adlarının `test` ile başlamasıdır; sınıfın test case sayılması için `TestCase`'ten kalıtım yeterlidir. Sınıf adını `Test` ile başlatmak bir adlandırma geleneğidir.

5.3 Assert metotları

En temel çalışma biçimi olan kara kutu sınamasında iç çalışmanın ayrıntısını bilmek gerekmez; verilen bir girdiye karşılık doğru sonucun üretilip üretilmediğine bakılır. Assert metotları bu senaryo üzerinden çalışır:

- `assertEquals(beklenen, sonuc)`: iki nesne birbirine eşitse test başarılıdır, değilse başarısızdır.
- `assertNotEquals(...)`: ters mantık; iki nesne eşit değilse test başarılıdır.
- `assertTrue(ifade)`: içine boolean döndüren bir hesaplama ya da metot çağrısı yazılır; ifade true ise test başarılıdır. `assertFalse` ters mantıktır.
- `assertNull(nesne)`: sonuç null ise test başarılıdır. `assertNotNull` tam tersidir.

> **Ders sonrası not:** Hoca `assertEquals` için iki nesnenin "aynı nesne" olması gerektiğini söyledi (00:25:48). `assertEquals` iki nesneyi `equals()` ile karşılaştırır, yani içerik eşitliğine bakar; iki referansın aynı nesneyi gösterip göstermediğini `assertSame` denetler. Dersin örneklerindeki String karşılaştırmaları da içerik eşitliğine dayanır.

5.4 setUp metodu

Testler yürütülmeden önce bir ortam hazırlanması gerekir: sorumluluklar hangi hazırlık verisi üzerinden sınanacaksa o veri oluşturulmalıdır. Bunun kodu `TestCase` sınıfındaki protected `setUp` metoduna yazılır; test sınıflarında bu metot override edilir. `setUp` programcı tarafından çağrılmaz; her test metodundan önce otomatik olarak çalışır.

Örneğin TestAppClass1'de beş test metodu varsa sıra şöyledir: `setUp`, birinci test metodu, `setUp`, ikinci test metodu, … Yani `setUp` bütün testlerden önce bir kez değil, her test metodundan önce yeniden çalışır.

> **Hocanın vurgusu:** Bu ayrıntıya dikkat edin. Bir şeyi sınarken test kodunun da doğru yazılması gerekir. Bir test başarısız olduğunda belki uygulama kodu doğrudur da test kodu yanlıştır; bununla da karşılaşılabilir.

Ek bilgi (derste anlatılmadı): Slaytta metodun adı `setup` diye yazılmış, ama dersteki kodda ve JUnit 3'te metodun adı `setUp`'tır. Java büyük/küçük harf duyarlı olduğundan `setup` adlı bir metot `TestCase.setUp`'ı override etmez ve otomatik çalışmaz.

Test sınıfı Eclipse'te sağ tıklanıp JUnit testi olarak çalıştırıldığında her test metodu sırayla çalıştırılır ve sonuçlar bildirilir. Test sınıflarını tek tek çalıştırmak zorunlu değildir; istenen kombinasyonda bir test suite içinde toplanabilirler. Bunun için içinde `suite` adlı bir metot bulunan bir sınıf yazılır (bkz. Bölüm 10.3).

6. JUnit 4.x Gelişmeleri

Slayttaki içerik şöyledir (ekrandan, 00:31:50):

- Geriye doğru uyumluluk korunmakla birlikte JUnit 4 sürümü ile annotation desteği gelmiştir (büyük/küçük harf duyarlılığı vardır).
  - Artık sınama sınıflarının `TestCase` sınıfından kalıtımla türetilmesi gerekmemektedir, ancak şu eklemeler yapılmalıdır: `import static org.junit.Assert.*;` ve `import org.junit.*;`
  - Artık sınama metotlarının adlarını `test` kelimesi ile başlatmak gerekli değildir; ilgili metotların başına `@Test` annotation koymak yeterlidir.
    * `setup` adlı metodun yerine ise `@Before` annotation gelmiştir.
  - Exception tanımlanan yazılımlarda atılması gereken exception'ların gerçekten ortaya çıkıp çıkmadığının sınanması da mümkün olmuştur.

Slayttaki kod örnekleri:

```
@Before
public void setUp() { /*Preparations*/ }
@Test
public void testSomething() { /*Do test*/ }

@Test(expected=SomeException.class)
public void testTheException() throws SomeException {
    doSomethingThatCreatesTheException();
}
```

Hocanın açıklamaları:

- Kalıtım zorunluluğunun kalkması bir avantajdır. Java'da çoklu kalıtım desteklenmediği için tek kalıtım hakkını `TestCase`'ten kalıtım almaya harcamak gerekmez.
- Hazırlık metoduna `@Before` koymak şartıyla `setUp` dışında bir ad (örneğin `initialize`) verilebilir. Annotation konmaz ama adı `setUp` olursa da sorun olmaz; JUnit 3'teki gibi geriye doğru uyumlu çalışır.
- Bir metodun başına `@Test` konursa adı `test` ile başlamasa da test case olarak algılanır ve yürütülür.
- Testin başarılı sayılması için bir exception atılması gerekiyorsa `@Test` annotation'ına parametre verilir: `expected=SomeException.class`. Bu durumda metodun içinde SomeException türünden bir exception ortaya çıkarsa test başarılı, çıkmazsa başarısız sayılır; her iki durumda da sıradaki test yürütülmeye devam eder.
- SomeException bir checked exception ise, yani normalde try-catch içinde denetlenmesi gereken ama test senaryosunda denetlenmeyecek bir exception ise, metodun tanımının sonunda `throws SomeException` beyan edilmelidir.
- Annotation'larda büyük/küçük harf duyarlılığı vardır.

  Ders sonrası not: Hoca JUnit 3'te bir exception fırlayınca bütün test vakalarının durduğunu söyledi (00:30:45). JUnit 3'te beklenmeyen bir exception, büyük olasılıkla, yalnız o test metodunu "error" olarak sonlandırır, diğer test metotları çalışmaya devam eder. JUnit 3'te beklenen bir exception'ı sınamak için genelde metot try-catch içine alınır ve exception atılmazsa `fail()` çağrılır; JUnit 4'teki `expected` parametresi bu kalıbı kısaltır.

Dersteki örnek senaryolarda exception atılması gerekmediği için hoca test kodlarını JUnit 3 üzerinden yazdı. JUnit 4'te test sınıf ve metot adlarının `test` ile başlaması zorunlu olmasa da

hoca, bu sınıf ve metotların test amaçlı olduğunu programcıya açıkça anlatacağı için yine `test` ile başlatılmasını daha iyi bulduğunu, bunun kişisel tercihi olduğunu söyledi.

7. Video Kiralama Örneği: Tasarım

7.1 Kaynak ve anlatım tarzı

Dersin bu kısmı Martin Fowler'ın Refactoring kitabından alınmıştır. Fowler kitabında, kod kusurları ve refactoring kataloğuna girmeden önce bu örneği verir: örneğin tasarımındaki kusurları gösterir, onları düzeltir, sonra her bir kod kusurunu ve onu gideren refactoring eylemlerini tek tek anlatır. Hoca ise önce teoriyi, yani belirli kod kusurlarını ve onları gideren refactoring eylemlerini anlatmayı tercih ettiğini söyledi. Bu temel örnek, ilerideki kod kusuru anlatımlarında kullanılan kod parçalarının bütününü içerir. Bu yüzden tasarımın ve kodun bilinmesi, sonraki anlatımları anlamayı kolaylaştırır.

7.2 Senaryo

Örnek bir video kaset kiralama bilgi sistemidir. Hoca, sektör kasetten DVD'ye, oradan akışa (streaming) geçse de "kiralayıp izleme" (pay per view) mantığının sürdüğünü, bu yüzden senaryonun geçerliliğini koruduğunu söyledi.

- Müşteriler (Customer) filmleri (Movie) ödünç alır.
- Üç film kategorisi vardır: normal filmler, çocuk filmleri ve yeni çıkan filmler. Her kategorinin kiralama ücreti ve süresi farklıdır: belli bir türde film için belli bir ücret ödeyerek belli sayıda gün kiralanır, sonraki her gün için ek ücret ödenir.
- Kiralama ve borç dökümü yapılır: hangi film kaç gün kiralandı ve bedeli ne kadar, satır satır yazılır; sonunda toplam borç yazılır.
- Kiralamalardan film türüne ve süreye göre bonus puan kazanılır. Makbuzda puan film başına değil, toplam olarak yazılır.

7.3 Sınıf şeması

Slayttaki sınıf şeması (ekrandan, 00:40:00):

```
Movie            Rental                Customer
-_priceCode:int  -_daysRented:int      +statement()

Customer 1 ---> * Rental
Rental   * ---> 1 Movie
```

Customer ile Movie arasında doğrudan ilişki kurulması doğru olmaz. Kaç gün kiralandığı bilgisi Movie sınıfında tutulursa, aynı film başka biri tarafından başka bir süre için kiralandığında bu bilgi bozulur. Kim neyi kiraladı dökümü, yani geçmiş hareketler saklanmak isteniyorsa araya ilişkiyi sağlayacak bir sınıf eklenmelidir. İş kiralama olduğu için bu sınıfın adı Rental'dır ve kiralama süresi (`_daysRented`) burada tutulur.

Fowler üye alanların adlarını alt çizgiyle (`_priceCode`, `_daysRented`) başlatmıştır. Hoca bunun amacını şöyle açıkladı: bir kodu bir yerde incelerken adı alt çizgiyle başlayan şeyin üye alan olduğu hemen anlaşılır. IDE'de bunun ek bir yararı yoktur, çünkü IDE üye alanları farklı renkle gösterir. Ama kod bir kitapta ya da ders notunda basılı olarak incelendiğinde büyük olasılıkla renklendirme yoktur; o zaman üye alanları metot parametrelerinden ayırt etmeyi kolaylaştırır.

7.4 statement() sıralama şeması

Slayttaki sıralama (sequence) şemasının metne dökümü şöyledir (ekrandan, 00:40:00). Katılımcılar: Kullanıcı, aCustomer, aRental, aMovie. Kaynak kodun yeri: `refactoring/fowler00`.

```
Kullanıcı -> aCustomer : statement()
  *[for all rentals]  (Tek kira için yapılan hesaplar her döngüde biriktirilir)
    aCustomer -> aRental : getMovie()        --> aMovie
    aCustomer -> aMovie  : getPriceCode()
    aCustomer -> aRental : getDaysRented()   (Tek kira için ücret hesabı tamamlanır)
```

```
    aCustomer -> aMovie  : getPriceCode()
    aCustomer -> aRental : getDaysRented()   (Tek kira için bonus hesabı tamamlanır)
    aCustomer -> aMovie  : getTitle()        (Tek kira için makbuz satırı basımı tamamlanır)
  (Biriken bilgiyle makbuz basımı tamamlanır)
```

`*[for all rentals]` gösterimi bir for-each döngüsüdür. Her kira kaydı için önce hangi filmin kiralandığı sorulur, sonra filmin fiyat kodu öğrenilir, sonra kaç gün kiralandığı sorulur ve bu kira işleminin bedeli hesaplanır. Bonus puan da fiyat kodu ve kiralama süresiyle ilgili olduğu için bu iki çağrı yinelenir ve bonus hesaplanır. Makbuz satırında film adı da gerektiği için `getTitle()` çağrılır; böylece "şu filmi şu kadar gün kiraladınız, borcunuz şu" bilgisi yazılabilir. Bonus kümülatif bir toplamda biriktirilir. Döngü bitince biriken bilgiyle toplam borç ve toplam puan eklenir ve `statement()` sona erer.

Hoca şemaya `statement()` çağrısının geri dönüş okunu çizmeyi unuttuğunu, not bileşenleri nedeniyle yer kalmadığını söyledi ve bunun neden önemli olduğunu açıkladı:

> **Hocanın vurgusu:** Sıralama şemalarında geri dönüş oklarının gösterilmesi önemlidir. Geri dönüş okları çizilmezse bir metot çağrısının bir öncekinin tamamlanmasından sonra mı başladığı, yoksa ikisinin paralel mi çalıştığı net anlaşılmaz.

8. Video Kiralama Örneği: Kodun İlk Hali

8.1 Paket düzeni

Kod, ders notlarıyla birlikte paylaşılan NDTM-kodlar projesindedir. `refactoring` klasöründe tasarımın ve kodun ilk hali `fowler_00` paketindedir. Her refactoring eyleminin ardından sınıflar yeni bir pakette toplanır: ilk eylemin sonucu `fowler_01`, ikinci eylemin sonucu `fowler_02` diye sürer (ekrandan, 01:00:00).

Test kodları ise `testing` adlı ayrı bir kaynak klasöründe, sınanan kodla aynı adlı pakette (`testing/fowler_00`, `testing/fowler_01`, …) durur. Böylece test kodu ile uygulama kodu fiziksel olarak ayrı dizinlerde, mantıksal olarak aynı pakettedir. Hoca bunu iki yolun avantajını birleştiren ("best of both worlds") bir teknik olarak gördü:

- Test kodu uygulama kodundan ayrılır.
- Aynı pakette olmak paket görünürlüğündeki üyelere erişim sağlar. Hoca paket görünürlüğündeki metotları çok önermediğini, ama bazen gerekebileceğini söyledi.
- Test kodunun uygulama kodunu import etmesi gerekmez. Ayrı paketlerde olsalardı, birden fazla sürüm bir arada bulunduğunda test senaryosunun doğru sürümü import edip etmediğinin ayrıca denetlenmesi gerekirdi.

Bu yapı Eclipse'te ve başka IDE'lerde de kurulabilir.

8.2 Customer sınıfı

Ekrandaki kod (ekrandan, 00:48:40; 00:50:00; 00:51:45):

```
package fowler_00;

import java.util.*;

public class Customer {
    private String _name;
    private Vector<Rental> _rentals = new Vector<Rental>();
    public Customer (String name){
        _name = name;
    }
    public void addRental(Rental arg) {
        _rentals.addElement(arg);
    }
    public String getName (){
        return _name;
    }
```

```
    public String statement( ) {
        double totalAmount = 0;
        int frequentRenterPoints = 0;
        Enumeration<Rental> rentals = _rentals.elements();
        String result = "Rental Record for " + ...
        while (rentals.hasMoreElements()) {
            double thisAmount = 0;
            Rental each = rentals.nextElement();
            //determine amounts for each line
            switch (each.getMovie().getPriceCode()) {
            case Movie.REGULAR:
                thisAmount += 2.0;
                if (each.getDaysRented() > 2)
                    thisAmount += (each.getDaysRented() - 2) * 1.5;
                break;
            case Movie.NEW_RELEASE:
                thisAmount += each.getDaysRented() * 3.0;
                break;
            case Movie.CHILDRENS:
                thisAmount += 1.5;
                if (each.getDaysRented() > 3)
                    thisAmount += (each.getDaysRented() - 3) * 1.5;
                break;
            }
            // add frequent renter points
            frequentRenterPoints ++;
            // add bonus for a two day new release rental
            if ((each.getMovie().getPriceCode() == Movie.NEW_RELEASE)
                    && each.getDaysRented() > 1) frequentRenterPoints ++;
            //show figures for this rental
            result += "\t" + each.getMovie().getTitle()+ "\t" +
            String.valueOf(thisAmount) + "\n";
            totalAmount += thisAmount;
        }
        ...
    }
}
```

Kod parçadır, derlenmedi. Satır 21'in sonu ve 50-56. satırlar ekrana gelmediği için `...` ile gösterildi. Testlerdeki beklenen metinlere göre makbuz şu satırlarla biter: `"Amount owed is " + toplam + "\n"` ve `"You earned " + puan + " frequent renter points"` (ekrandan, 01:02:55).

> **Ders sonrası not:** Hoca kira kayıtlarının Java'nın jenerik veri yapılarından bir bağlı listede (linked list) tutulduğunu söyledi (00:48:28). Ekrandaki kodda kayıtlar `Vector<Rental>` içinde tutuluyor, `addElement` ile ekleniyor ve `elements()` ile alınan bir `Enumeration` ile dolaşılıyor (ekrandan, 00:48:40, 00:50:00). Hoca da bu sırada Vector'ün thread-safe olup olmadığını hatırlayamadığını söyledi.
>
> **Ek bilgi (derste anlatılmadı):** `java.util.Vector` senkronize (thread-safe) bir listedir; `ArrayList` ise senkronize değildir. `Enumeration`, `Iterator`'dan önceki eski dolaşım arayüzüdür. Örneğin bu eski sınıfları kullanması, büyük olasılıkla, Fowler'ın kitabının ilk baskısının eski Java sürümleriyle yazılmış olmasındandır.

`statement()` metodunun işleyişi şöyledir:

- Toplam borç `totalAmount`, toplam puan `frequentRenterPoints` adlı yerel değişkenlerde biriktirilir.
- Makbuz metni `"Rental Record for "` ve müşteri adıyla başlar.
- Her kira kaydı için o kaydın bedeli `thisAmount` adlı geçici değişkende tutulur. Sıradaki kaydın filmi ve filmin fiyat kodu alınır ve bir switch-case yapısına verilir.
- Fiyatlandırma:

  - Normal film (REGULAR): 2.0 ödenir, bu 2 gün kiralama hakkı verir; 2 günü aşan her gün için 1.5 daha ödenir.
  - Yeni çıkan film (NEW_RELEASE): her gün için 3.0 ödenir.
  - Çocuk filmi (CHILDRENS): 1.5 ödenir, bu 3 gün kiralama hakkı verir; 3 günü aşan her gün için 1.5 daha ödenir.
- Her kiralama 1 puan kazandırır. Yeni çıkan bir film 1 günden uzun süre kiralanmışsa 1 ekstra puan daha kazanılır.
- Bir tab, film adı, bir tab ve bedelden oluşan makbuz satırı eklenir; bedel toplama eklenir.
- Döngü bitince toplam borç ve kazanılan puan yazılır ve `result` döndürülür.

Hoca para biriminin dolar olması gerektiğini düşündüğünü söyledi; derste anlatırken "lira" dedi.

> **Hocanın vurgusu:** `statement()` metodu imzasıyla birlikte 17. satırdan 56. satıra kadar uzanır. Metotların bu kadar uzun olması tercih edilmez; bu en bariz kod kusurudur. İkinci bariz kusur switch-case içinde türe göre farklı işlem yapılmasıdır. Bunlar sırayla, yavaş yavaş refactor edilecektir.

8.3 Rental ve Movie sınıfları

Rental, bir müşteriyi bir filme bağlayan basit bir sınıftır: film ve kaç gün kiralandığı bilgisini tutar. Kurucusu ikisini birden alır; getter metotları vardır, setter'a ihtiyaç duyulmamıştır.

Movie sınıfının başı şöyledir (ekrandan, 00:52:40):

```
package fowler_00;

public class Movie {
    public static final int CHILDRENS = 2;
    public static final int REGULAR = 0;
    public static final int NEW_RELEASE = 1;
    private String _title;
    private int _priceCode;
    public Movie(String title, int priceCode) {
        _title = title;
        _priceCode = priceCode;
    }
    public int getPriceCode() {
        return _priceCode;
    }
    public void setPriceCode(int arg) {
    ...
```

Fiyat kodları `public static final int` sabitlerle (0, 1, 2) tanımlanmıştır. Fiyat kodunun setter'ı vardır, çünkü yeni çıkan bir film altı ay ya da bir yıl sonra normal film haline düşmelidir. Başlığın (`_title`) setter'ı yoktur, yalnız getter'ı vardır.

9. Video Kiralama Örneğinin JUnit Testleri

9.1 TestMovie

Movie küçücük bir sınıftır; "bunun nesini test edeceğiz" denebilir. Ama madem farklı film türleri var, türlerin gerçekten farklı fiyat kodlarına sahip olduğu sınanabilir (ekrandan, 00:57:20):

```
public class TestMovie extends TestCase {

    public void testMovie( ) {
        Movie matrix = new Movie("The Matrix",Movie.REGULAR);
        Movie monster = new Movie("Monsters, Inc.",Movie.CHILDRENS);
        Movie surrogate = new Movie("Surrogates", Movie.NEW_RELEASE);
        assertTrue("Bunlar farklı tür filmlerdir",
                matrix.getPriceCode() != monster.getPriceCode() );
        assertTrue("Bunlar farklı tür filmlerdir",
```

```
                matrix.getPriceCode() != surrogate.getPriceCode() );
        assertFalse("Bunlar farklı tür filmlerdir",
                monster.getPriceCode() == surrogate.getPriceCode() );
    }
}
```

Bu sınıf için `junit.framework.TestCase` import edilir. Burada `setUp` yoktur. Bir test metodunda tek assert önerilir, ama birden fazla olabilir; bu durumda `testMovie`'nin başarılı sayılması için üç assert'in de geçmesi gerekir, biri bile başarısız olursa test başarısız sayılır. Üç farklı film türü için üç ikili karşılaştırma yapılır. Assert metotlarının ilk String parametresi, test başarısız olursa ekrana yazılacak açıklamadır. Üçüncü assert'te mantık terstir: Monsters, Inc. ile Surrogates'in fiyat kodları eşitse bu bir hatadır, bu yüzden `assertFalse` kullanılır.

Sınıf JUnit testi olarak çalıştırılınca yeşil çubuk görülür: tek test metodu başarılıdır.

9.2 TestCustomer: hazırlık

Asıl sınanması gereken, Customer sınıfının `statement()` metodudur: farklı türlerden filmler farklı sürelerle kiralandığında doğru bedeller ve doğru puanlar mı yazılıyor? Bunun için en az bir müşteri ve her türden birer film tanımlanır, filmler değişik sürelerle kiralanır (ekrandan, 01:00:00):

```
package fowler_00;

import junit.framework.TestCase;

public class TestCustomer extends TestCase {
    private Customer yunus;
    private Movie matrix, monster, surrogate, terminator;

    protected void setUp() {
        yunus = new Customer("Yunus Emre Selçuk");
        matrix = new Movie("The Matrix",Movie.REGULAR);
        monster = new Movie("Monsters, Inc.",Movie.CHILDRENS);
        surrogate = new Movie("Surrogates", Movie.NEW_RELEASE);
        terminator = new Movie("Terminator Salvation",Movie.NEW_RELEASE);
    }
```

Müşteri adı olarak hoca kendi adını kullanmıştır. `setUp` bir müşteri ve dört film oluşturur (bir normal, bir çocuk, iki yeni film), ama filmleri müşteriye kiralatmaz. Kiralamalar her testte farklı olacağı için test metotlarının içinde yapılır. `setUp` her test metodundan önce yeniden çalıştığı için yalnız bütün test vakalarında ortak olan hazırlık, yani ilgili nesnelerin oluşturulması burada yapılır.

9.3 testGetName ve testStatementWhenEmpty

```
    public void testGetName( ) {
        String sonuc = yunus.getName( );
        assertEquals("Yunus Emre Selçuk",sonuc);
    }

    public void testStatementWhenEmpty( ) {
        String sonuc = yunus.statement();
        String beklenen = "Rental Record for Yunus Emre Selçuk\n";
        beklenen += "Amount owed is 0.0\n";
        beklenen += "You earned 0 frequent renter points";
        assertEquals(beklenen, sonuc);
    }
```

(ekrandan, 01:02:55)

`testGetName` çok basit görünebilir; ama hoca bununla çalışılan platformun Türkçe karakterleri destekleyip desteklemediğinin de sınandığını söyledi, çünkü müşteri adında Türkçe karakter

vardır.

`testStatementWhenEmpty`, müşteri hiç film kiralamamışken boş makbuz basımını sınar. `setUp`'ta kiralama yapılmadığı için müşteri şu an bu durumdadır. Makbuz `statement()` ile elde edilir, sonra boş makbuzun nasıl olması gerektiği elle yazılır: hiç film satırı yok, toplam borç 0, kazanılan puan 0. İki metin `assertEquals` ile karşılaştırılır.

9.4 Uzun ve kısa süreli kiralama testleri

Her türden bir filmin, ek ücret gerektirecek kadar uzun kiralandığı test (ekrandan, 01:04:10; beklenen değerler 01:07:25'teki hata izinden):

```
public void testStatementWithMoviesLongRent( ) {
    yunus.addRental( new Rental(matrix, 3) );
    yunus.addRental( new Rental(monster, 4) );
    yunus.addRental( new Rental(surrogate, 2) );
    String sonuc = yunus.statement();
    String beklenen = "Rental Record for Yunus Emre Selçuk\n";
    beklenen += "\tThe Matrix\t3.5\n";
    beklenen += "\tMonsters, Inc.\t3.0\n";
    beklenen += "\tSurrogates\t6.0\n";
    beklenen += "Amount owed is 12.5\n";
    beklenen += "You earned 4 frequent renter points";
    assertEquals(beklenen, sonuc);
}
```

37-39. satırların ortası ekranda bir ipucu kutusunun altında kalmıştı. Değerler, testin başarısız olduğu andaki hata izinde görünen beklenen metinden alındı (ekrandan, 01:07:25).

Hoca test adının dilbilgisi açısından kusursuz olmasına değil, tam olarak neyin sınandığını anlatmasına dikkat ettiğini söyledi. Beklenen değerlerin elle hesabı:

- The Matrix (normal, 3 gün): $2.0 + (3 - 2) \times 1.5 = 3.5$
- Monsters, Inc. (çocuk, 4 gün): $1.5 + (4 - 3) \times 1.5 = 3.0$
- Surrogates (yeni, 2 gün): $2 \times 3.0 = 6.0$
- Toplam: $3.5 + 3.0 + 6.0 = 12.5$
- Puan: her kiralama 1 puan, toplam 3; Surrogates yeni film ve 1 günden uzun kiralandığı için +1 bonus; toplam 4.

  Ders sonrası not: Hoca anlatırken normal filmin dört gün, yeni filmin iki gün kiralandığını söyledi (01:03:49). Ekrandaki kodda The Matrix 3 gün, Monsters, Inc. 4 gün, Surrogates 2 gün kiralanıyor (ekrandan, 01:04:10).

Kısa süreli kiralama testi, ek ücret gerektirmeyen sürelerle aynı şeyi sınar (ekrandan, 01:04:45):

```
public void testStatementWithMoviesShortRent( ) {
    yunus.addRental( new Rental(matrix, 2) );
    yunus.addRental( new Rental(monster, 3) );
    yunus.addRental( new Rental(surrogate, 1) );
    String sonuc = yunus.statement();
    String beklenen = "Rental Record for Yunus Emre Selçuk\n";
    beklenen += "\tThe Matrix\t2.0\n";
    beklenen += "\tMonsters, Inc.\t1.5\n";
    beklenen += "\tSurrogates\t3.0\n";
    beklenen += "Amount owed is 6.5\n";
    beklenen += "You earned 3 frequent renter points";
    assertEquals(beklenen, sonuc);
}
```

Elle hesap: normal film 2 gün için yalnız taban ücret 2.0; çocuk filmi 3 gün için yalnız taban ücret 1.5; yeni film 1 gün için $1 \times 3.0 = 3.0$; toplam 6.5. Yeni film yalnız 1 gün kiralandığı için bonus yoktur; puan 3.

9.5 Yeni film bonusu testi

Yeni filmlerde ekstra bonus kazanıldığı için iki yeni film kiralanır: biri 2 gün, biri 1 gün (ekrandan, 01:05:25):

```
    public void testNewReleaseRentalBonus( ) {
        yunus.addRental( new Rental(surrogate, 2) );
        yunus.addRental( new Rental(terminator, 1) );
        String sonuc = yunus.statement();
        String beklenen = "Rental Record for Yunus Emre Selçuk\n";
        beklenen += "\tSurrogates\t6.0\n";
        beklenen += "\tTerminator Salvation\t3.0\n";
        beklenen += "Amount owed is 9.0\n";
        beklenen += "You earned 3 frequent renter points";
        assertEquals(beklenen, sonuc);
    }
```

Hocanın anlatımıyla: 2 günlük kiralamadan 2 puan, 1 günlük kiralamadan 1 puan beklenir; toplam 3 puan. Bedeller $2 \times 3.0 = 6.0$ ve $1 \times 3.0 = 3.0$, toplam 9.0.

TestCustomer JUnit testi olarak çalıştırılınca yeşil çubuk görülür; ağaç açıldığında beş test metodunun beşinin de başarılı olduğu ayrı satırlarda görünür.

Bütün beklenen değerler python ile yeniden hesaplanarak doğrulandı (12.5 / 4 puan, 6.5 / 3 puan, 9.0 / 3 puan).

10. Başarısız Bir Testin İncelenmesi ve Test Suite

10.1 Bilerek hata yapmak

Hoca bir sorun olsaydı ne olacağını göstermek için uygulama koduna bilerek bir hata koydu. Gereksinim belgesine göre normal filmlerde 2 günü aşan her gün için 1.5 ödenmesi gerekirken bunu yanlışlıkla 1.0 yapmış gibi davrandı (ekrandan, 01:07:25):

```
            case Movie.REGULAR:
                thisAmount += 2.0;
                if (each.getDaysRented() > 2)
                    thisAmount += (each.getDaysRented() - 2) * 1.0;
                break;
```

TestCustomer yeniden çalıştırıldığında çubuk kırmızı olur. JUnit panelindeki özet: Runs 5/5, Errors 0, Failures 1. Başarısız test `testStatementWithMoviesLongRent`'tir. Birden fazla sorunlu test varsa JUnit ilk sorunlu test metoduna gider.

10.2 Hata izini okumak

Panelin altındaki "Failure Trace" bölümünde `junit.framework.ComparisonFailure` görünür. JUnit beklenen ile elde edilen metni yan yana verir ve iki metnin ayrışmaya başladığı ilk noktadan ayrışmanın bittiği yere kadar olan kısmı köşeli parantez içine alır (ekrandan, 01:07:25):

```
expected:<... The Matrix 3.[5
Monsters, Inc. 3.0
Surrogates 6.0
Amount owed is 12.5]
You earned 4 freque...>
but was:<... The Matrix 3.[0
Monsters, Inc. 3.0
Surrogates 6.0
Amount owed is 12.0]
You earned 4 freque...>
```

Yani The Matrix satırında 3.5 beklenirken 3.0 elde edilmiştir; toplam da 12.5 yerine 12.0 olmuştur. Hesap: $2.0 + (3 - 2) \times 1.0 = 3.0$ ve $3.0 + 3.0 + 6.0 = 12.0$.

Hoca JUnit olmasaydı yalnız “test başarısız” bilgisinin görüleceğini, nedenini bulmak için metinleri ekrana yazdırıp farkı kendimizin aramak zorunda kalacağımızı söyledi. JUnit bu işi üstlenip programcıyı doğrudan sorunun olduğu yere götürür.

Farkın The Matrix satırında olduğu görülünce normal film kodu açılır, gereksinim belgesine bakılır: “2 liraya 2 gün, sonra her gün için 1.5”. Kodda 1.0 yazıldığı görülür ve düzeltilir.

> **Ders sonrası not:** Hoca hata izine bakarken bir an “yeni bir film”, yani yeni film kiralamasıyla ilgili kodda yanlışlık olduğunu söyledi (01:08:37), hemen ardından normal film (REGULAR) koduna baktı. Farkın görüldüğü The Matrix normal bir filmdir; hata da REGULAR dalındaki çarpandadır.

Testler yeniden çalıştırılırken Eclipse’in JUnit panelindeki “Rerun Test - Failures First” düğmesi kullanılabilir. Hoca açıklamanın “only” değil “first” dediğine dikkat çekti: önce başarısız olan testler, sonra zaten geçen testler çalıştırılır. Testler kısa olduğu için hepsi hızlıca çalışır ve yeşil çubuk geri gelir.

10.3 TestAll ile test suite

Test suite oluşturmak için şöyle bir sınıf yazılır (ekrandan, 01:10:00):

```
package fowler_00;

import junit.framework.*;

public class TestAll {
    public static Test suite( ) {
        TestSuite suite = new TestSuite("Bütün testler");
        suite.addTestSuite(TestCustomer.class); //reflection API
        suite.addTestSuite(TestMovie.class);
        return suite;
    }
}
```

- `public static Test suite()` imzası sabittir; metodun adı `suite` olmak zorundadır.
- Bir `TestSuite` nesnesi oluşturulur; String parametresiyle istenen ad verilir.
- Suite’e eklenecek her test sınıfı `addTestSuite(SınıfAdı.class)` ile verilir.
- Suite döndürülünce istenen sayıda test sınıfı tek bir koşturmayla çalıştırılabilir.

`.class` ile sınıfın verilmesi Java’nın Reflection API’siyle ilgilidir. Hoca bu API’yi şöyle tanıttı: herhangi bir sınıfın byte kodunu salt okunur biçimde inceleyip bir nesnenin hangi sınıfa ve pakete ait olduğunu, hangi üye alanları ve metotları bulunduğunu, metotların parametrelerini tek tek sorgulamayı sağlar. Bu konu dersin kapsamında değildir; isteyenler inceleyebilir.

Örnek Sorular

Soru 1 (derste sözlü, 00:11:25): Refactoring ne zaman yapılmalıdır? Bunun gerektiği nasıl anlaşılır?

Cevap: Hocanın cevabı: kod derlenip toparlanmaya gerek duyduğunda. Bu, koda yeni özellik eklemek ya da hata düzeltmek gibi bakım işlerinde gitgide daha çok zorlanmaya başlandığında; bir hatanın kök nedenini bulmak zorlaştığında ya da bir hatayı düzeltmek için birçok ayrı yeri kurcalamak gerektiğinde anlaşılır. Proje yönetim yazılımında aynı büyüklükteki işlerin tamamlanma süresinin uzaması da bir işarettir. Kısaca, kod kötü kokmaya başladığında.

Soru 2 (derste sözlü, 00:20:12): Bir sınıfın bir sorumluluğu, henüz kodlanmamış başka sınıflarla birlikte çalışmayı gerektiriyorsa o sınıfın birim sınaması bekletilmeli midir?

Cevap: Hocanın cevabı: hayır. Henüz yazılmamış sınıfların yerine derleme hatası vermeyecek minimum bir iskelet (vekil, sahte, yalancı kod; stub, dummy, surrogate, proxy) yazılır. Örneğin bir denetim yapıp boolean döndüren bir metodun gövdesine `return true;` yazılarak o tarafta hata çıkmadığı varsayılır ve kendi kodumuzun sınamaları yürütülür. Vekil kod, gerçek sınıflar gerçeklenene dek kullanılır.

Soru 3 (derste sözlü, 00:57:06): Yalnız bir başlık ve bir fiyat kodu tutan Movie gibi küçük bir sınıfta ne sınanabilir?

Cevap: Hocanın cevabı: farklı film türleri olduğu için, üç türden birer film oluşturup fiyat kodlarının birbirinden farklı olduğu sınanabilir. `TestMovie.testMovie` bunu iki `assertTrue` (`!=`) ve bir `assertFalse` (`==`) ile yapar (ekrandan, 00:57:20).

Soru 4 (ekrandan, 01:04:10 ve 01:07:25): Bir müşteri The Matrix'i (normal) 3 gün, Monsters, Inc.'i (çocuk) 4 gün, Surrogates'i (yeni) 2 gün kiralıyor. `statement()` makbuzunda hangi bedeller, toplam borç ve puan yazılmalıdır? Normal film için 2 günü aşan gün ücreti yanlışlıkla 1.0 kodlanırsa sonuç nasıl değişir?

Cevap: Testteki beklenen değerler: The Matrix $2.0+1\times1.5=3.5$, Monsters, Inc. $1.5+1\times1.5=3.0$, Surrogates $2\times3.0=6.0$; toplam borç 12.5; puan $3+1=4$ (yeni film 1 günden uzun kiralandığı için bonus). Hatalı kodla The Matrix $2.0+1\times1.0=3.0$ olur, toplam 12.0'a düşer; puan değişmez. JUnit bu farkı `expected:<...3.[5 ... 12.5]...> but was:<...3.[0 ... 12.0]...>` biçiminde gösterir.

Soru 5 (ekrandan, 01:05:25): Surrogates (yeni) 2 gün, Terminator Salvation (yeni) 1 gün kiralanırsa makbuzda ne yazmalıdır?

Cevap: Testteki beklenen değerler: Surrogates $2\times3.0=6.0$, Terminator Salvation $1\times3.0=3.0$, toplam 9.0. Puan: 2 günlük kiralama 1 + 1 bonus = 2, 1 günlük kiralama 1; toplam 3.

Dersin Toparlaması

Bu derste refactoring, dışarıdan görünen davranışı koruyarak iç yapıyı değiştirmek olarak tanımlandı. Amacı anlaşılabilirlik, esneklik, yeniden kullanılabilirlik, yüksek uyum ve düşük bağlaşım gibi iç kalite özelliklerini iyileştirmektir. Refactoring çevik bir anlayışla, küçük adımlarla ve sürekli sınamayla yapılır; kod "kötü kokmaya" başladığında, yani code smell'ler belirdiğinde gerekir. Code smell bir hata değil, kodu iyileştirme gerektiğinin işaretidir ve bir refactoring eylemi yeni kokulara yol açabilir.

Sürekli sınamanın aracı olarak birim sınaması ve JUnit anlatıldı: JUnit 3'te `TestCase`'ten kalıtım, `test` önekli metotlar, her testten önce çalışan `setUp`, assert metotları ve `TestAll.suite()`; JUnit 4'te `@Test`, `@Before` ve `@Test(expected=...)`. Fowler'ın video kiralama örneğinin ilk hali (`fowler_00`) ve onun JUnit testleri incelendi, bilerek konan bir hatanın JUnit'te nasıl görüldüğü gösterildi. Hocaya göre uzun `statement()` metodu ve switch-case yapısı bu kodun en bariz kusurlarıdır. Sonraki derslerde ayrıntılı kod kusurlarına, bunların nasıl fark edileceğine ve bu örnek üzerinde adım adım nasıl düzeltileceğine geçilecektir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.